

Activity - Estimating Lifetime

From `6_regression.ipynb` : build a Kaplan-Meier survival analysis on the Telco churn data and compare lifetime between payment methods.

survival term	Telco column
duration T	<code>tenure</code> , months as a customer
event E	<code>Churn == "Yes"</code>
censored	still a customer - we only know they lasted <i>at least</i> <code>tenure</code>
group	<code>PaymentMethod</code>

```
In [1]: from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from lifelines import KaplanMeierFitter
from lifelines.statistics import multivariate_logrank_test, pairwise_logrank_test
from lifelines.utils import restricted_mean_survival_time

pd.set_option("display.precision", 3)

df = pd.read_csv("Telco-Churn.csv")
df["T"] = df["tenure"].astype(float)
df["E"] = (df["Churn"] == "Yes").astype(int)

print(df.shape)
df[["customerID", "PaymentMethod", "tenure", "Churn", "T", "E"]].head()
```

(7043, 23)

```
Out[1]:
```

	customerID	PaymentMethod	tenure	Churn	T	E
0	7590-VHVEG	Electronic check	1	No	1.0	0
1	5575-GNVDE	Mailed check	34	No	34.0	0
2	3668-QPYBK	Mailed check	2	Yes	2.0	1
3	7795-CFOCW	Bank transfer (automatic)	45	No	45.0	0
4	9237-HQITU	Electronic check	2	Yes	2.0	1

1. What is in the data

Four things worth knowing before fitting anything.

```
In [2]: raw = pd.read_csv("Telco-Churn.csv")

print(f"rows {len(raw)}    columns {raw.shape[1]}")
print(f"duplicate customerID {raw.customerID.duplicated().sum()}")
print(f"duplicate rows      {raw.duplicated().sum()}")
print(f"null cells          {raw.isna().sum().sum()}")
print()

# TotalCharges arrives as text, not a number - something in it is not numeric.
not_numeric = pd.to_numeric(raw["TotalCharges"], errors="coerce").isna()
print(f"TotalCharges dtype    {raw['TotalCharges'].dtype}")
print(f" non-numeric rows      {not_numeric.sum()}")
print(f" their tenure values {sorted(raw.loc[not_numeric,
'tenure'].unique())}")
```

```
rows 7043    columns 21
duplicate customerID 0
duplicate rows      0
null cells          0

TotalCharges dtype    object
non-numeric rows      11
their tenure values [np.int64(0)]
```

So the one dirty column and the customers with no follow-up time are the same eleven rows: a customer who signed up this month has been billed nothing yet. That settles what to do with them in the next section.

Does every value sit in a range that makes sense?

```
In [3]: numbers = raw.assign(TotalCharges=pd.to_numeric(raw["TotalCharges"],
errors="coerce"))
numbers.select_dtypes("number").agg(["min", "max", "median"]).T
```

```
Out[3]:
```

	min	max	median
SeniorCitizen	0.00	1.00	0.000
tenure	0.00	72.00	29.000
MonthlyCharges	18.25	118.75	70.350
TotalCharges	18.80	8684.80	1397.475

```
In [4]: # Categoricals: the check is that no level is a typo, a blank, or a stray
"unknown".
SUSPECT = {"", " ", "na", "n/a", "null", "none", "unknown", "?", "-"}

for col in raw.select_dtypes("object").columns.drop("customerID"):
    levels = sorted(raw[col].unique())
    flagged = [v for v in levels if str(v).strip().lower() in SUSPECT]
```

```

folded = len({str(v).strip().lower() for v in levels}) != len(levels)
note = f"  <-- {flagged}" if flagged else ("  <-- differs only by
case/space" if folded else "")
print(f"{col:<18} {len(levels)} levels{note}")

```

```

gender          2 levels
Partner         2 levels
Dependents      2 levels
PhoneService    2 levels
MultipleLines   3 levels
InternetService 3 levels
OnlineSecurity  3 levels
OnlineBackup     3 levels
DeviceProtection 3 levels
TechSupport     3 levels
StreamingTV      3 levels
StreamingMovies  3 levels
Contract        3 levels
PaperlessBilling 2 levels
PaymentMethod    4 levels
TotalCharges    6531 levels  <-- [' ']
Churn           2 levels

```

```

In [5]: # Cross-column checks: do the columns agree with each other?
services = ["OnlineSecurity", "OnlineBackup", "DeviceProtection",
            "TechSupport", "StreamingTV", "StreamingMovies"]

no_internet = raw["InternetService"] == "No"
mismatched = sum(((raw[c] == "No internet service") != no_internet).sum() for c
in services)
print(f'rows where a service flag disagrees with InternetService:
{mismatched}')
print(f'rows where MultipleLines disagrees with PhoneService:      '
      f'{{{(raw["MultipleLines"] == "No phone service") != (raw["PhoneService"]
== "No"))}.sum()}}')
print()

# TotalCharges should be roughly tenure * MonthlyCharges - unless the price
changed.
billed = numbers.dropna(subset=["TotalCharges"])
implied = billed["tenure"] * billed["MonthlyCharges"]
drift = (billed["TotalCharges"] - implied) / implied

print(f"TotalCharges vs tenure x MonthlyCharges")
print(f"  median drift  {drift.median():+.3f}")
print(f"  range        {drift.min():+.3f} to {drift.max():+.3f}")
print(f"  highest bill  {billed['TotalCharges'].max():.2f}, "
      f"against {72 * numbers['MonthlyCharges'].max():.2f} at today's top price
for 72 months")

```

rows where a service flag disagrees with InternetService: 0
rows where MultipleLines disagrees with PhoneService: 0

TotalCharges vs tenure x MonthlyCharges

median drift +0.000
range -0.311 to +0.573
highest bill 8684.80, against 8550.00 at today's top price for 72 months

Everything sits where it should: SeniorCitizen is 0/1, tenure runs 0-72, no charge is negative or zero, no categorical column carries a typo or a blank level, and the service flags agree with InternetService and PhoneService on every row.

The one thing the range check turns up is in TotalCharges. Its maximum is larger than 72 months at the highest monthly price on record, and while it matches tenure x MonthlyCharges at the median it drifts either way for individuals - because MonthlyCharges is today's price and people change plans. Two consequences: the column is not redundant, but it also must stay out of the model in section 7, since a total bill is largely a restatement of how long someone has been paying, which is the very thing being predicted.

```
In [6]: counts = raw["tenure"].value_counts().sort_index()
middle = raw.loc[raw["tenure"].between(2, 71), "tenure"]

print(f"tenure = 1 {counts.get(1)} customers (just signed up)")
print(f"tenure = 72 {counts.get(72)} customers (the edge of the observation window)")
print(f"median of everything in between: {middle.median():.0f} months")
```

```
tenure = 1 613 customers (just signed up)
tenure = 72 362 customers (the edge of the observation window)
median of everything in between: 31 months
```

```
In [7]: band = pd.cut(raw["tenure"], [-1, 6, 12, 24, 48, 72],
                      labels=["0-6", "7-12", "13-24", "25-48", "49-72"])
churn_by_band = (raw.assign(churned=(raw["Churn"] == "Yes").astype(int))
                 .groupby(band, observed=True)
                 .agg(customers=("churned", "size"), churn_rate=("churned",
                                                                "mean")))

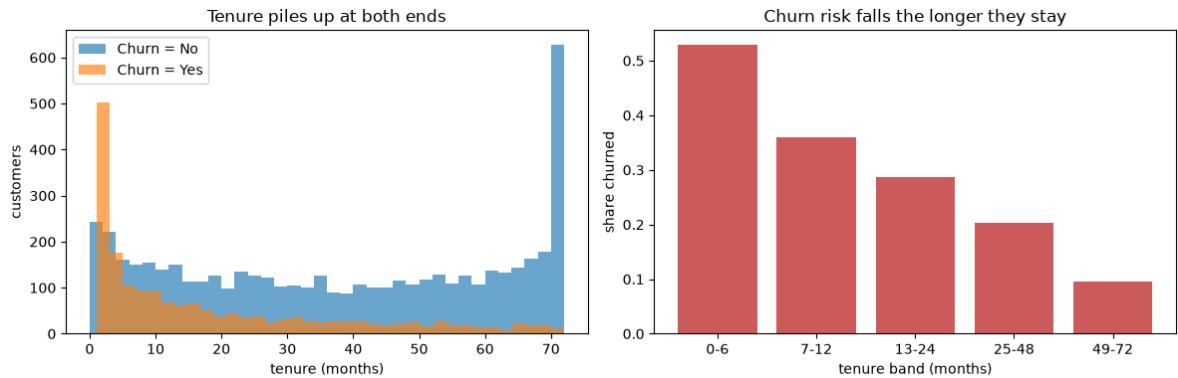
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for label, sub in raw.groupby("Churn"):
    axes[0].hist(sub["tenure"], bins=36, alpha=0.65, label=f"Churn = {label}")
axes[0].set(xlabel="tenure (months)", ylabel="customers",
            title="Tenure piles up at both ends")
axes[0].legend()

axes[1].bar(churn_by_band.index.astype(str), churn_by_band["churn_rate"],
            color="indianred")
axes[1].set(xlabel="tenure band (months)", ylabel="share churned",
            title="Churn risk falls the longer they stay")
```

```
plt.tight_layout()
Path("figures").mkdir(exist_ok=True)
fig.savefig("figures/eda_tenure.png", dpi=150, bbox_inches="tight")
plt.show()

churn_by_band.round(3)
```



Out[7]:

	customers	churn_rate
tenure		
0-6	1481	0.529
7-12	705	0.359
13-24	1024	0.287
25-48	1594	0.204
49-72	2239	0.095

Two things to carry forward. The spike at 72 months is the edge of the observation window, not a natural lifetime - those customers are censored by when the data was collected, which is exactly what Kaplan-Meier is built to handle. And churn risk is not constant: it runs at 53% in the first six months and 10% after four years. A falling hazard is a warning that the Cox model's proportional-hazards assumption may not hold, which is tested in section 7.

In [8]:

```
# Payment method is not assigned at random - it travels with contract type.
pd.crosstab(raw["PaymentMethod"], raw["Contract"], normalize="index").round(3)
```

Out[8]:

	Contract	Month-to-month	One year	Two year
PaymentMethod				
Bank transfer (automatic)		0.381	0.253	0.365
Credit card (automatic)		0.357	0.261	0.382
Electronic check		0.782	0.147	0.071
Mailed check		0.554	0.209	0.237

Electronic-check customers are on month-to-month contracts 78% of the time against 36% for credit card. Any lifetime gap between payment methods could be contract type in disguise, so section 7 fits a model that holds contract constant.

2. Censoring

The point of Kaplan-Meier: most rows are *censored*, not observed failures. Averaging `tenure` would treat "still here after 60 months" as a lifetime of 60, which understates every group.

```
In [9]: print(f"customers          {len(df)}")
print(f"churned (events)      {df.E.sum()} ({df.E.mean():.1%}")
print(f"still active          {(1 - df.E).sum()} (right-censored)")
print(f"tenure == 0            {(df['T'] == 0).sum()} (brand new, no follow-up
time)")
print()

# A zero-length observation carries no information for the estimator and
# lifelines
# warns about it, so those rows come out.
df = df[df["T"] > 0].copy()
print(f"kept {len(df)} rows")
```

```
customers          7043
churned (events)    1869 (26.5%)
still active        5174 (right-censored)
tenure == 0         11 (brand new, no follow-up time)
```

kept 7032 rows

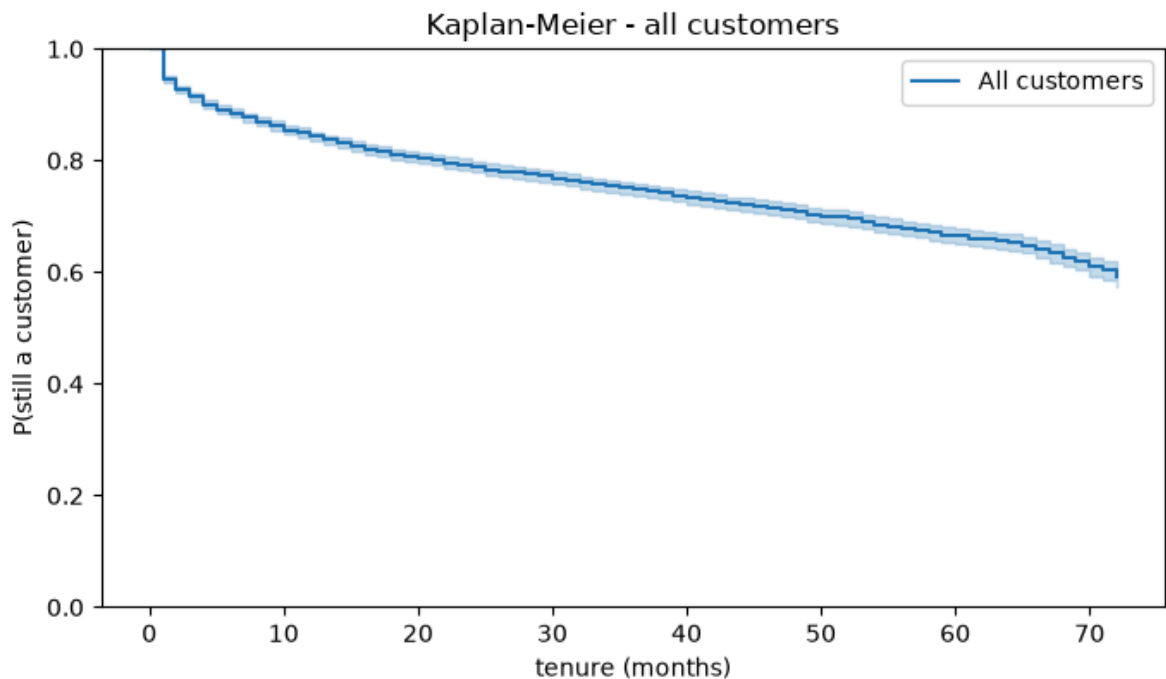
3. Overall survival curve

```
In [10]: kmf = KaplanMeierFitter()
kmf.fit(df["T"], event_observed=df["E"], label="All customers")

ax = kmf.plot_survival_function(figsize=(7, 4.2))
ax.set(xlabel="tenure (months)", ylabel="P(still a customer)",
       title="Kaplan-Meier - all customers", ylim=(0, 1))
```

```
plt.tight_layout()
plt.show()

print("median survival time:", kmf.median_survival_time_)
print(kmf.survival_function_at_times([12, 24, 48, 72]).round(3))
```



```
median survival time: inf
12    0.843
24    0.789
48    0.709
72    0.593
Name: All customers, dtype: float64
```

4. By payment method

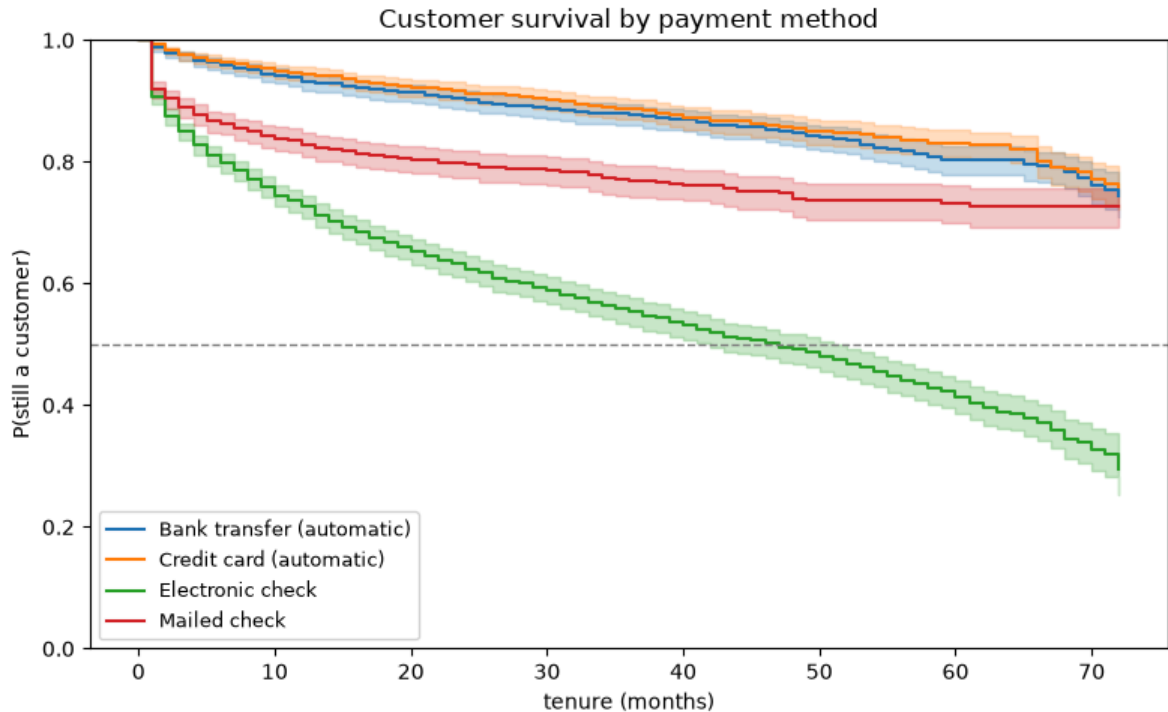
```
In [11]: METHODS = sorted(df["PaymentMethod"].unique())
fitters = {}

fig, ax = plt.subplots(figsize=(8, 5))
for method in METHODS:
    group = df[df["PaymentMethod"] == method]
    f = KaplanMeierFitter().fit(group["T"], event_observed=group["E"],
label=method)
    fitters[method] = f
    f.plot_survival_function(ax=ax, ci_show=True)

ax.set(xlabel="tenure (months)", ylabel="P(still a customer)",
       title="Customer survival by payment method", ylim=(0, 1))
ax.axhline(0.5, color="grey", lw=1, ls="--")
ax.legend(loc="lower left", fontsize=9)

plt.tight_layout()
Path("figures").mkdir(exist_ok=True)
```

```
fig.savefig("figures/km_by_payment_method.png", dpi=150, bbox_inches="tight")
plt.show()
```



5. Comparing lifetime

`median_survival_time_` is the statistic the lecture uses, but three of the four curves never reach 0.5 inside the 72 months observed, so it comes back as `inf` for them. Two statistics that do exist for every group:

- **survival at fixed times** - the share still subscribed at 12, 24, 48, 72 months
- **RMST** (restricted mean survival time) - the area under the curve up to a chosen horizon, read as "expected months as a customer during the first N"

RMST needs that horizon picked by hand, and the choice is not neutral: over a short window everybody survives most of it, so the groups look closer together. Reporting several horizons keeps that visible. Twelve months is the one to quote - the risk set is still 55-87% full there, where at 72 months it is down to 1-10% and the curve is carried by a handful of customers.

```
In [12]: HORIZONS = [12, 24, 48, 72]
HEADLINE = 12

rows = []
for method, f in fitters.items():
    s = f.survival_function_at_times(HORIZONS).to_numpy()
    row = {
        "n": int(f.event_observed.shape[0]),
        "churned": int(f.event_observed.sum()),
```

```

        "median": f.median_survival_time_,
    }
    row |= {f"S({h})": v for h, v in zip(HORIZONS, s)}
    row |= {f"RMST({h})": restricted_mean_survival_time(f, t=h) for h in
HORIZONS}
    rows.append({"PaymentMethod": method} | row)

summary =
pd.DataFrame(rows).set_index("PaymentMethod").sort_values(f"RMST({HEADLINE})")
summary

```

Out[12]:

	n	churned	median	S(12)	S(24)	S(48)	S(72)	RMST(12)	RMST(24)
--	---	---------	--------	-------	-------	-------	-------	----------	----------

PaymentMethod

Electronic check	2365	1071	47.0	0.727	0.624	0.493	0.295	9.863	17.9
Mailed check	1604	308	inf	0.828	0.797	0.740	0.726	10.545	20.2
Bank transfer (automatic)	1542	258	inf	0.931	0.902	0.847	0.745	11.561	22.5
Credit card (automatic)	1521	232	inf	0.945	0.913	0.855	0.758	11.623	22.7

In [13]:

```

# How much of each horizon does a customer of this group actually spend
subscribed?
share = pd.DataFrame({
    f"{h} mo": summary[f"RMST({h})"] / h for h in HORIZONS
})
share.loc["gap (best - worst)"] = share.max() - share.min()
share

```

Out[13]:

	12 mo	24 mo	48 mo	72 mo
--	-------	-------	-------	-------

PaymentMethod

Electronic check	0.822	0.748	0.653	0.573
Mailed check	0.879	0.845	0.808	0.783
Bank transfer (automatic)	0.963	0.941	0.909	0.875
Credit card (automatic)	0.969	0.949	0.918	0.886
gap (best - worst)	0.147	0.201	0.265	0.313

6. Are the differences real?

The log-rank test compares the curves without assuming any shape for them.

In [14]:

```

overall = multivariate_logrank_test(df["T"], df["PaymentMethod"], df["E"])
print(f"all four groups: chi2 = {overall.test_statistic:.1f}, p =

```

```
{overall.p_value:.3g}")
print()

pairwise = pairwise_logrank_test(df["T"], df["PaymentMethod"], df["E"])
pairwise.summary[["test_statistic", "p"]].round(4)
```

all four groups: chi2 = 865.2, p = 3.07e-187

Out[14]:

		test_statistic	p
Bank transfer (automatic)	Credit card (automatic)	0.868	0.351
	Electronic check	510.035	0.000
	Mailed check	51.072	0.000
Credit card (automatic)	Electronic check	539.740	0.000
	Mailed check	64.815	0.000
Electronic check	Mailed check	152.456	0.000

7. Cox proportional hazards

The curves above compare payment methods without holding anything else constant, and section 1 showed payment method is tangled with contract type. A Cox model separates the two.

The coefficients are read as **hazard ratios**, `exp(coef)` : the multiplier on the instantaneous churn rate relative to the reference level, at every point in time. HR = 2 means twice the churn rate of the reference; HR = 1 means no difference.

Reference levels: credit card (automatic), two-year contract.

```
In [15]: from lifelines import CoxPHFitter
from lifelines.statistics import proportional_hazard_test

REFERENCE = ["PaymentMethod_Credit card (automatic)", "Contract_Two year"]

design = pd.get_dummies(df[["PaymentMethod", "Contract"]],
dtype=float).drop(columns=REFERENCE)
design["tenure"] = df["T"].to_numpy()
design["churned"] = df["E"].to_numpy()

payment_cols = [c for c in design.columns if c.startswith("PaymentMethod")]
design.head()
```

Out[15]:

	PaymentMethod_Bank transfer (automatic)	PaymentMethod_Electronic check	PaymentMethod_Mailed check	Contract_I to-
0	0.0	1.0	0.0	
1	0.0	0.0	1.0	
2	0.0	0.0	1.0	
3	1.0	0.0	0.0	
4	0.0	1.0	0.0	

Model A - payment method alone

In [16]:

```
cox_a = CoxPHFitter().fit(design[payment_cols + ["tenure", "churned"]],
                           "tenure", "churned")

cox_a.summary[["exp(coef)", "exp(coef) lower 95%", "exp(coef) upper 95%", "p"]]
```

Out[16]:

	exp(coef)	exp(coef) lower 95%	exp(coef) upper 95%	p
covariate				
PaymentMethod_Bank transfer (automatic)	1.089	0.912	1.301	3.446e-01
PaymentMethod_Electronic check	4.621	4.005	5.332	1.130e-97
PaymentMethod_Mailed check	2.149	1.810	2.551	2.540e-18

Model B - payment method adjusted for contract type

In [17]:

```
cox_b = CoxPHFitter().fit(design, "tenure", "churned")

print(f"concordance model A {cox_a.concordance_index_:.3f} model B  
{cox_b.concordance_index_:.3f}")

cox_b.summary[["exp(coef)", "exp(coef) lower 95%", "exp(coef) upper 95%", "p"]]
```

concordance model A 0.676 model B 0.809

Out[17]:

	exp(coef)	exp(coef) lower 95%	exp(coef) upper 95%	p
covariate				
PaymentMethod_Bank transfer (automatic)	1.028	0.861	1.228	7.593e-01
PaymentMethod_Electronic check	2.104	1.822	2.430	3.981e-24
PaymentMethod_Mailed check	1.848	1.555	2.197	3.447e-12
Contract_Month-to-month	54.102	39.712	73.706	3.566e-141
Contract_One year	6.888	4.944	9.598	4.068e-30

In [18]:

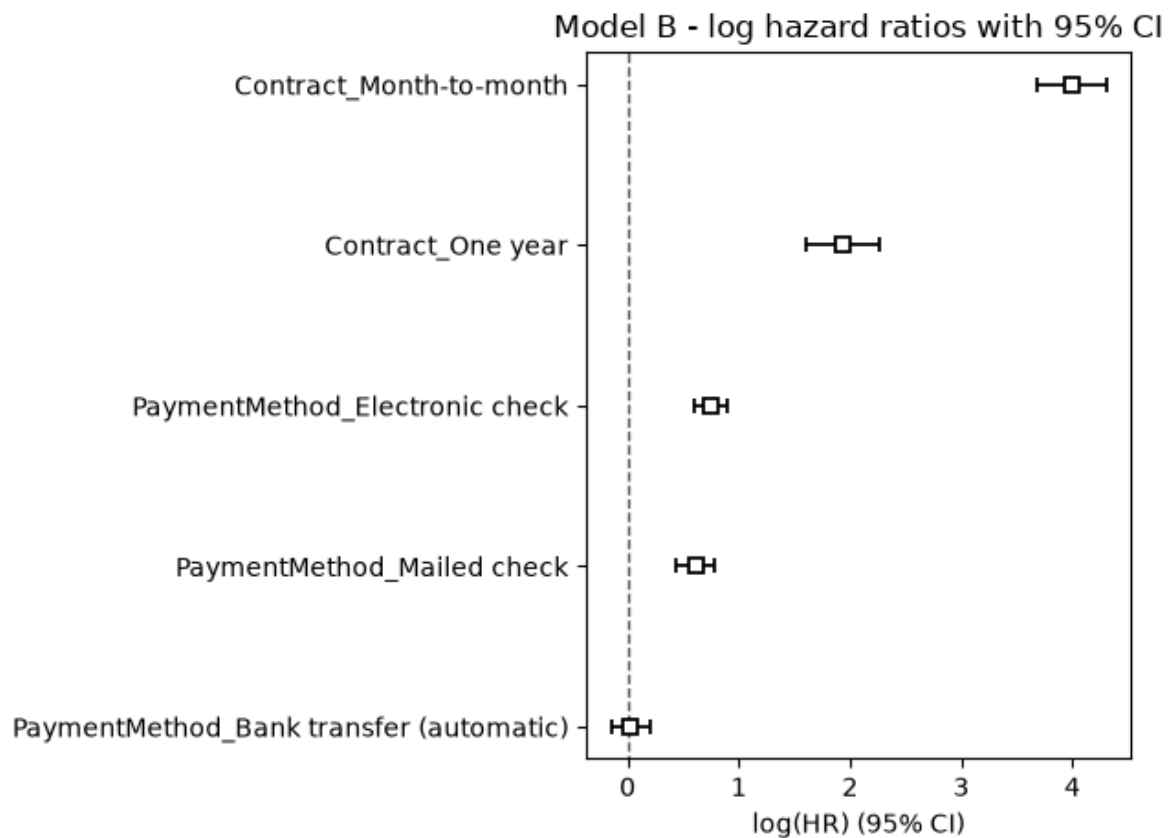
```
comparison = pd.DataFrame({
    "HR alone (A)": cox_a.hazard_ratios_,
    "HR adjusted (B)": cox_b.hazard_ratios_[payment_cols],
})
comparison["shrinkage"] = 1 - comparison["HR adjusted (B)"] / comparison["HR alone (A)"]
comparison
```

Out[18]:

	HR alone (A)	HR adjusted (B)	shrinkage
covariate			
PaymentMethod_Bank transfer (automatic)	1.089	1.028	0.056
PaymentMethod_Electronic check	4.621	2.104	0.545
PaymentMethod_Mailed check	2.149	1.848	0.140

In [19]:

```
ax = cox_b.plot()
ax.set_title("Model B - log hazard ratios with 95% CI")
plt.tight_layout()
plt.savefig("figures/cox_hazard_ratios.png", dpi=150, bbox_inches="tight")
plt.show()
```



Does the model's own assumption hold?

Cox assumes a covariate's hazard ratio is *constant over time*. That is testable, and here it fails for some covariates - so each HR above should be read as an average effect across the 72 months, not a fixed multiplier. The mailed-check curve in section 3 shows why: it drops steeply in the first months and then flattens.

```
In [20]: proportional_hazard_test(cox_b, design,
time_transform="rank").summary[["test_statistic", "p"]]
```

```
Out[20]:
```

	test_statistic	p
Contract_Month-to-month	20.686	5.411e-06
Contract_One year	0.008	9.286e-01
PaymentMethod_Bank transfer (automatic)	0.303	5.821e-01
PaymentMethod_Electronic check	9.082	2.582e-03
PaymentMethod_Mailed check	30.595	3.180e-08

8. Minimum follow-up time

Two different questions hide under "how long do we need to watch customers".

How soon can a conclusion be drawn? Censor every customer administratively at month `t` - as if the study had stopped there - and re-run the log-rank test. The smallest `t` whose test comes back significant is the minimum follow-up that particular comparison needs.

How long do the estimates stay trustworthy? A survival curve is only as good as the number of customers still at risk behind it. Once the risk set thins out, the curve is carried by a handful of people and the confidence band widens accordingly.

```
In [21]: FOLLOWUPS = range(1, 25) # months of follow-up to try
T_all, E_all, G_all = df["T"].to_numpy(), df["E"].to_numpy(),
df["PaymentMethod"].to_numpy()

first_significant = {}
for t in FOLLOWUPS:
    censored_T = np.minimum(T_all, t)
    censored_E = ((E_all == 1) & (T_all <= t)).astype(int)
    for pair, p in pairwise_logrank_test(censored_T, G_all,
censored_E).summary["p"].items():
        if pair not in first_significant and p < 0.05:
            first_significant[pair] = (t, p)

full = pairwise_logrank_test(T_all, G_all, E_all).summary["p"]
rows = []
for pair in full.index:
    t, p = first_significant.get(pair, (np.nan, np.nan))
    rows.append({"pair": " vs ".join(pair),
                "min follow-up (months)": t,
                "p at that point": p,
                "p on full 72 months": full[pair]})

pd.DataFrame(rows).sort_values("min follow-up (months)").set_index("pair")
```

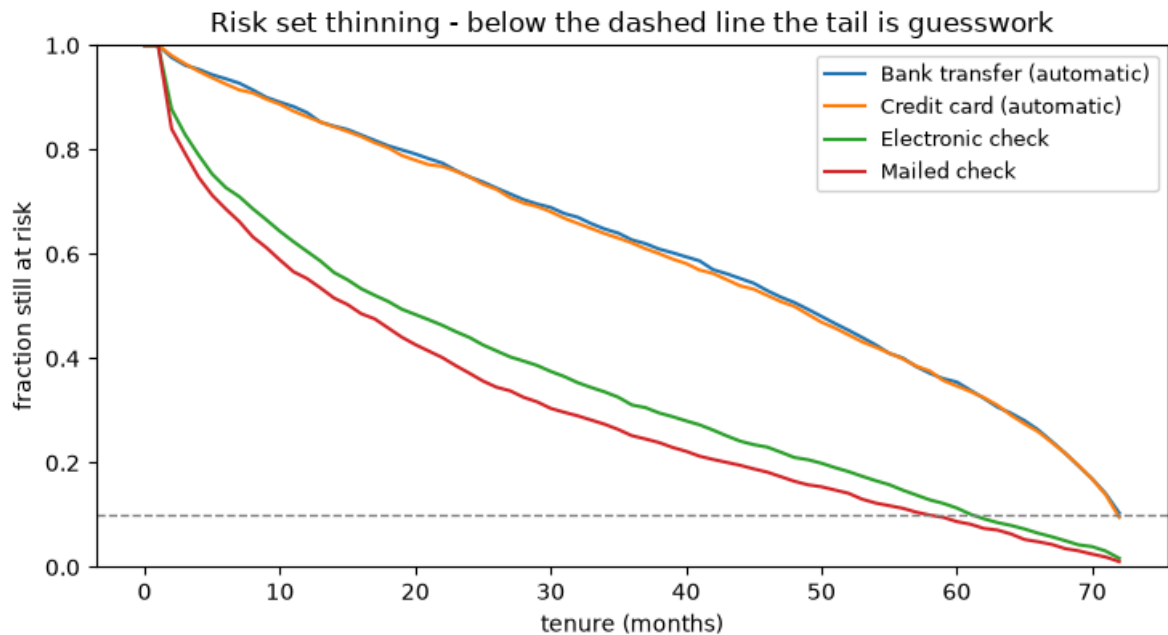
Out[21]:

	min follow-up (months)	p at that point	p on full 72 months
pair			
Bank transfer (automatic) vs Electronic check	1.0	5.673e-25	6.231e-113
Bank transfer (automatic) vs Mailed check	1.0	1.930e-19	8.905e-13
Credit card (automatic) vs Electronic check	1.0	1.765e-28	2.148e-119
Credit card (automatic) vs Mailed check	1.0	4.866e-23	8.226e-16
Electronic check vs Mailed check	2.0	5.579e-03	5.038e-35
Bank transfer (automatic) vs Credit card (automatic)	NaN	NaN	3.514e-01

```
In [22]: at_risk = pd.DataFrame({
    method: [(group["T"] >= t).mean() for t in (12, 24, 36, 48, 60, 72)]
    for method, group in df.groupby("PaymentMethod")
}, index=[f"t={t}" for t in (12, 24, 36, 48, 60, 72)].T
at_risk.columns.name = "fraction still at risk"

fig, ax = plt.subplots(figsize=(7.5, 4.2))
for method, group in df.groupby("PaymentMethod"):
    months = np.arange(0, 73)
    ax.plot(months, [(group["T"] >= t).mean() for t in months], lw=1.5,
label=method)
ax.axhline(0.10, color="grey", ls="--", lw=1)
ax.set(xlabel="tenure (months)", ylabel="fraction still at risk",
    title="Risk set thinning - below the dashed line the tail is guesswork",
ylim=(0, 1))
ax.legend(fontsize=9)
plt.tight_layout()
plt.savefig("figures/at_risk_thinning.png", dpi=150, bbox_inches="tight")
plt.show()

at_risk
```



Out[22]:

	fraction still at risk	t=12	t=24	t=36	t=48	t=60	t=72
Bank transfer (automatic)		0.871	0.748	0.627	0.507	0.355	0.104
Credit card (automatic)		0.863	0.748	0.621	0.500	0.347	0.095
Electronic check		0.605	0.440	0.311	0.210	0.113	0.017
Mailed check		0.553	0.372	0.252	0.165	0.087	0.011

9. Conclusion

```
In [23]: best, worst = summary.index[-1], summary.index[0]

print(f"longest-lived group {best}")
print(f"shortest-lived group {worst}")
print()
for h in HORIZONS:
    b, w = summary.loc[best, f"RMST({h})"], summary.loc[worst, f"RMST({h})"]
    print(f"over {h:>2} months: {b:5.2f} vs {w:5.2f} -> gap {b - w:5.2f}
months ({(b - w) / h:.1%} of the window)")
print()
print(f"electronic-check hazard ratio, unadjusted
{cox_a.hazard_ratios_['PaymentMethod_Electronic check']:.2f}")
print(f"electronic-check hazard ratio, given contract
{cox_b.hazard_ratios_['PaymentMethod_Electronic check']:.2f}")
```

longest-lived group Credit card (automatic)
shortest-lived group Electronic check

over 12 months: 11.62 vs 9.86 -> gap 1.76 months (14.7% of the window)
over 24 months: 22.78 vs 17.95 -> gap 4.82 months (20.1% of the window)
over 48 months: 44.05 vs 31.32 -> gap 12.73 months (26.5% of the window)
over 72 months: 63.81 vs 41.24 -> gap 22.57 months (31.3% of the window)

electronic-check hazard ratio, unadjusted 4.62
electronic-check hazard ratio, given contract 2.10

Lifetime by payment method. Over a first year, a credit-card customer spends 11.6 of the 12 months subscribed against 9.9 for an electronic-check customer - 97% of the window versus 82%. The two automatic methods cannot be told apart at all: the log-rank test gives $p = 0.35$ on the full data, and it never falls below 0.05 at any follow-up horizon. Mailed check sits between them. Median survival time, the statistic the lecture uses, only exists for electronic check (47 months) because the other three curves never fall to 0.5, so RMST does the comparing here.

The gap grows with the window. It is 1.8 months over a year, 12.7 months over four years, 22.6 months over the full six - and as a share of the window it climbs from 15% to 31%. So a single horizon flatters or exaggerates depending on where it is cut. Twelve months is the honest one to quote because the risk set is still full there; the longer figures are real but rest on a thinning tail.

How much of that is the payment method itself. On its own, electronic check carries a hazard ratio of 4.6 against credit card. Once contract type is held constant it falls to about 2.1 - so roughly half the apparent effect was contract type wearing a payment method's clothes. What is left is still a doubled churn rate, and mailed check behaves much the same way once adjusted, which points at the manual methods themselves rather than at who chooses them.

Minimum follow-up. Every comparison the data can support is already significant after **2 months** of follow-up: four of the five pairs separate within the first month, and electronic check versus mailed check needs a second. Watching longer sharpens the estimates but changes no conclusion - except for automatic bank transfer versus credit card, which stays indistinguishable however long anyone waits. At the other end, fewer than 10% of the manual-payment customers are still at risk past 60 months, so the tail of those curves should not be quoted.